

0.0.1. Beispiel: LINQ von Nath

Alle Bücher und Pages aus Tabelle Books laden **Library Context**

```
public class LibraryContext : DbContext {
    // ctor
    // ...
    public virtual DbSet<Book> Books{ get; set; } // virtual wichtig für lazy loading
    public virtual DbSet<Page> Pages{ get; set; }
}
// In Entities Collection Property: public virtual ICollection<Page> Pages { get; set; }
```

Version 1

```
List<Book> buecher = new ();
await using (LibraryContext context = new()) {
    buecher = await context.Books
        .Include(b => b.Pages) // Eager Loading
        .ToListAsync();
} // Verbindung wird getrennt
```

Mit LINQ

```
await using (LibraryContext context = new()) {
    var q1 = from b in context.Books
    where b.Author == "Nath"
    orderby b.Name
    select new { b.Name, b.Preis };
}

await using (LibraryContext context = new()) {
    var q2 = from b in context.Books
    where b.Length > 600
    join p in context.Pages on b.Id equals p.BookId
    group p by p.PageNumber
    into g
    select new { Num = g.Key, g.Count() };
}
```

Add und Delete mit Lazy Loading

```
await using (LibraryContext context = new()) {
    var badBooks = context.Books.Where(b => b.Rating ≤ 3); // noch kein Zugriff auf DB
    foreach (Book b in badBooks) { // DB-Zugriff
        foreach (Page p in b.Pages) { // DB-Zugriff
            // V1 als "zum löschen" markiert, Referenzen können in C# noch vorhanden sein.
            context.Entry(p).State = EntityState.Deleted;
        }

        b.Pages.Clear(); // V2 nur C# Collection Property, nicht DB ausser bei cascading evt.

        b.Pages.Add(new Page{ content = "lol", PageNumber=0});

        // alle Pages löschen (1+N Query Problem) :
        context.Pages.RemoveRange(b.Pages);
    }
    await context.SaveChangesAsync(); // DB-Zugriff
}
```

Alle Pages löschen mit Eager Loading

```
var pagesToDelete =
    from b in context.Books
    where b.Rating ≤ 3
    from p in b.Pages
    select p;

context.Pages.RemoveRange(pagesToDelete);
```

refl. code für s. 73

```
var bfType = typeof(BugfixAttribute);
if (method.IsDefined(bfType)) {
    BugfixAttribute b = (BugfixAttribute)method
        .GetCustomAttributes(bfType)
        .Single();
    return b.BugId;
}
return 0

// Gemäss Internet gibt es eine besser Variante
var bugfix
= method.GetCustomAttributes<BugfixAttribute>();
return bugfix?.BugId ?? 0;
// mit Vererbung:
// ..GetCustomAttributes<..>(inherit: true);
```